

编译原理第三次实验测试用例：目录

1	A 组测试用例	3
1.1	A-1	3
1.2	A-2	4
1.3	A-3	5
1.4	A-4	6
1.5	A-5	7
2	B 组测试用例	8
2.1	B-1	8
2.2	B-2	9
2.3	B-3	11
3	C 组测试用例	12
3.1	C-1	12
3.2	C-2	13
4	E 组测试用例	14
4.1	E1-1	15
4.2	E1-2	15
4.3	E1-3	16
4.4	E2-1	18
4.5	E2-2	19
4.6	E2-3	20
5	结束语	22

1 A 组测试用例

本组测试用例共 5 个，均为比较简单的程序，简单检查针对赋值/算术语句、分支语句、循环语句、数组表达式和函数调用的翻译。

1.1 A-1

1.1.1 输入

```
1 int main() {  
2     int n;  
3     int prod;  
4     int acc;  
5     n = 5;  
6     prod = 1;  
7     acc = 0;  
8     while (n > 0) {  
9         prod = prod * n;  
10        acc = acc + prod;  
11        n = n - 1;  
12    }  
13    write(prod);  
14    write(acc);  
15    return 0;  
16 }
```

1.1.2 输出

```
1 >>> Empty  
2 [120, 325]
```

1.1.3 说明

测试 while 循环：从 $n=5$ 递减，维护阶乘累乘 `prod` 与部分阶乘和 `acc`，最后两次 `write`。程序无 `read`。

输出中以 `>>>` 起始的行表示程序输入（`Empty` 表示无输入）；其后一行方括号内为各次 `write` 的输出序列，整数之间以逗号分隔。实际评测时每个 `read/write` 各占一行，此处为节省版面写在同一行（下同）。

1.2 A-2

1.2.1 输入

```
1 int main() {
2     int a;
3     int b;
4     int c;
5     int t;
6     a = read();
7     b = read();
8     c = read();
9     t = 0;
10    if (a > b) {
11        t = t + 1;
12    }
13    if (b > c) {
14        t = t + 2;
15    }
16    if (a > c) {
17        t = t + 4;
18    }
19    write(t);
20    if (a + b + c > 100) {
21        write(1000);
22    } else {
23        write(2000);
24    }
25    if ((a * b) / 2 > c) {
26        write(3000);
27    } else {
28        write(4000);
29    }
30    return 0;
31 }
```

1.2.2 输出

```
1 >>> 40, 30, 20
2 [7, 2000, 3000]
3
4 >>> 10, 50, 8
5 [6, 2000, 3000]
6
7 >>> 60, 50, 5
8 [7, 1000, 3000]
```

1.2.3 说明

测试三次 read 后，多段 if 与比较、算术组合：先根据两两大小关系给 t 累加编码，再按和是否超过 100、以及 $(a*b)/2$ 与 c 的关系输出不同常数。

记号说明同 A-1。

1.3 A-3

1.3.1 输入

```
1 int main() {
2     int buf[6];
3     int i;
4     int k;
5     i = 0;
6     while (i < 6) {
7         buf[i] = i * i + 3;
8         i = i + 1;
9     }
10    k = read();
11    write(buf[0]);
12    write(buf[k]);
13    write(buf[5]);
14    return 0;
15 }
```

1.3.2 输出

```

1 >>> 2
2 [3, 7, 28]
3
4 >>> 4
5 [3, 19, 28]
6
7 >>> 0
8 [3, 3, 28]

```

1.3.3 说明

测试一维数组初始化 ($\text{buf}[i]=i*i+3$) 与下标 read: 输出首元素、下标 k 处元素与末尾元素。

记号说明同 A-1。

1.4 A-4

1.4.1 输入

```

1 int main() {
2     int n;
3     int i;
4     int j;
5     int sum;
6     n = read();
7     sum = 0;
8     i = 1;
9     while (i <= n) {
10         j = 1;
11         while (j <= n) {
12             if ((i / 2) * 2 != i) {
13                 sum = sum + i * j;
14             }
15             j = j + 1;
16         }
17         i = i + 1;
18     }
19     write(sum);

```

```

20     return 0;
21 }

```

1.4.2 输出

```

1 >>> 3
2 [24]
3
4 >>> 2
5 [3]

```

1.4.3 说明

测试二重 while: 对 $1 \leq i, j \leq n$, 仅在 i 为奇数 ($(i/2)*2 \neq i$) 时把 $i*j$ 累加到 sum。
记号说明同 A-1。

1.5 A-5

1.5.1 输入

```

1 int fib(int n) {
2     if (n <= 1) {
3         return n;
4     }
5     return fib(n - 1) + fib(n - 2);
6 }
7
8 int main() {
9     int x;
10    x = read();
11    write(fib(x));
12    return 0;
13 }

```

1.5.2 输出

```

1 >>> 6
2 [8]

```

```
3
4 >>> 5
5 [5]
6
7 >>> 0
8 [0]
```

1.5.3 说明

测试递归函数 fib: fib(0)=0, fib(1)=1, 否则为前两项之和。
记号说明同 A-1。

2 B 组测试用例

本组测试用例共 3 个, 较 A 组测试用例复杂, 这里不专门针对赋值和算术语句设计测试用例。

2.1 B-1

2.1.1 输入

```
1 int scoreAt(int val) {
2     return val * 2 + 1;
3 }
4
5 int main() {
6     int vals[6];
7     int i;
8     int best;
9     int bestIdx;
10    i = 0;
11    while (i < 6) {
12        vals[i] = read();
13        i = i + 1;
14    }
15    best = vals[0];
16    bestIdx = 0;
17    i = 1;
18    while (i < 6) {
```

```

19         if (vals[i] > best) {
20             best = vals[i];
21             bestIdx = i;
22         }
23         i = i + 1;
24     }
25     write(bestIdx);
26     write(best);
27     write(scoreAt(best));
28     return 0;
29 }

```

2.1.2 输出

```

1 >>> 3, 8, 2, 8, 1, 0
2 [1, 8, 17]
3
4 >>> 5, 4, 3, 9, 9, 2
5 [3, 9, 19]
6
7 >>> 1, 2, 3, 4, 5, 0
8 [4, 5, 11]

```

2.1.3 说明

测试长度为6的数组、六次 read、手写求最大值下标,以及形参为整型的辅助函数 scoreAt (对最大值返回 val*2+1)。

记号说明同 A-1。

2.2 B-2

2.2.1 输入

```

1 int pairVal(int a, int b) {
2     int s;
3     s = a + b;
4     return s * s + a;
5 }

```

```

6
7 int main() {
8     int thr;
9     int cnt;
10    int i;
11    int j;
12    thr = read();
13    cnt = 0;
14    i = 0;
15    while (i < 4) {
16        j = i + 1;
17        while (j < 4) {
18            if (pairVal(i, j) > thr) {
19                cnt = cnt + 1;
20            }
21            j = j + 1;
22        }
23        i = i + 1;
24    }
25    write(cnt);
26    return 0;
27 }

```

2.2.2 输出

```

1 >>> 5
2 [4]
3
4 >>> 20
5 [1]

```

2.2.3 说明

测试二重循环枚举 $0 \leq i < j < 4$, 调用 `pairVal ((i+j)*(i+j)+i)`, 统计大于阈值 `thr` 的对数。

记号说明同 A-1。

2.3 B-3

2.3.1 输入

```
1 int clampAdd(int x, int lim) {
2     int y;
3     y = x + 3;
4     if (y > lim) {
5         return lim;
6     }
7     return y;
8 }
9
10 int main() {
11     int buf[5];
12     int len;
13     int i;
14     int prod;
15     len = read();
16     if (len > 5) {
17         len = 5;
18     }
19     if (len < 1) {
20         len = 1;
21     }
22     i = 0;
23     while (i < 5) {
24         buf[i] = i + 2;
25         i = i + 1;
26     }
27     i = 0;
28     prod = 1;
29     while (i < len) {
30         buf[i] = clampAdd(buf[i] * 2, 20);
31         prod = prod * buf[i];
32         i = i + 1;
33     }
34     write(prod);
```

```

35     write(buf[len - 1]);
36     return 0;
37 }

```

2.3.2 输出

```

1 >>> 1
2 [7, 7]
3
4 >>> 3
5 [693, 11]
6
7 >>> 5
8 [135135, 15]

```

2.3.3 说明

测试辅助函数 `clampAdd`、定长数组前缀上的变换与乘积：读入长度 `len`（裁剪到 `[1,5]`），对前 `len` 个元素做 `clampAdd(buf[i]*2,20)` 并累乘，再输出末尾被改元素。
记号说明同 A-1。

3 C 组测试用例

本组测试用例共 2 个，是较经典的问题。

3.1 C-1

3.1.1 输入

```

1 int main() {
2     int a;
3     int b;
4     int t;
5     a = read();
6     b = read();
7     if (a < b) {
8         t = a;
9         a = b;
10        b = t;

```

```

11     }
12     while (b != 0) {
13         t = a - (a / b) * b;
14         a = b;
15         b = t;
16     }
17     write(a);
18     return 0;
19 }

```

3.1.2 输出

```

1 >>> 48, 18
2 [6]
3
4 >>> 100, 35
5 [5]
6
7 >>> 17, 13
8 [1]

```

3.1.3 说明

测试辗转相除求最大公约数：若 $a < b$ 先交换，再 while ($b \neq 0$) 用整除与减法求余。
记号说明同 A-1。

3.2 C-2

3.2.1 输入

```

1 int digitSum(int n) {
2     int s;
3     int x;
4     s = 0;
5     x = n;
6     if (x < 0) {
7         x = 0 - x;
8     }

```

```

9      while (x > 0) {
10          s = s + (x - (x / 10) * 10);
11          x = x / 10;
12      }
13      return s;
14  }
15
16  int main() {
17      int v;
18      v = read();
19      write(digitSum(v));
20      return 0;
21  }

```

3.2.2 输出

```

1  >>> 12345
2  [15]
3
4  >>> 909
5  [18]
6
7  >>> 7
8  [7]
9
10 >>> 0
11 [0]

```

3.2.3 说明

测试辅助函数 digitSum: 对非负绝对值逐位求和（负输入先取相反数）。
记号说明同 A-1。

4 E 组测试用例

本组测试用例共 6 个，针对不同分组进行测试。

E1 组针对 3.1 分组测试结构体的翻译，E2 组针对 3.2 分组测试一维数组作为参数和高维数组的翻译。每组 3 个测试用例。

4.1 E1-1

4.1.1 输入

```
1 struct Rect {
2     int width;
3     int height;
4 };
5
6 int main() {
7     struct Rect r;
8     int area;
9     int halfPerim;
10    r.width = 6;
11    r.height = 7;
12    area = r.width * r.height;
13    halfPerim = r.width + r.height;
14    write(area);
15    write(halfPerim);
16    return 0;
17 }
```

4.1.2 输出

```
1 >>> Empty
2 [42, 13]
```

4.1.3 说明

测试简单结构体 Rect：固定宽高，输出面积与半周长 (width+height)。无 read。
针对选做 **要求 4.1** 的同学；未实现结构体时应提示无法翻译且不输出中间代码。
记号说明同 A-1。

4.2 E1-2

4.2.1 输入

```
1 struct Pair {
2     int first;
```

```

3     int second;
4 };
5
6 int mix(struct Pair p) {
7     return p.first * p.second - p.first;
8 }
9
10 int main() {
11     struct Pair q;
12     int outVal;
13     q.first = read();
14     q.second = read();
15     outVal = mix(q);
16     write(outVal);
17     return 0;
18 }

```

4.2.2 输出

```

1 >>> 4, 5
2 [16]
3
4 >>> 10, 3
5 [20]

```

4.2.3 说明

测试结构体作形参: mix 返回 $p.first * p.second - p.first$ 。
 针对选做 **要求 4.1** 的同学; 未实现时应提示无法翻译且不输出中间代码。
 记号说明同 A-1。

4.3 E1-3

4.3.1 输入

```

1 struct Inner {
2     int p;
3     int q;

```

```

4 };
5
6 struct Wrapper {
7     struct Inner u;
8     int tag;
9 };
10
11 int evalWrap(struct Wrapper wrap) {
12     int s;
13     s = 0;
14     if (wrap.tag > 0) {
15         s = s + wrap.u.p;
16     }
17     if (wrap.tag < 10) {
18         s = s + wrap.u.q * 2;
19     }
20     return s;
21 }
22
23 int main() {
24     struct Wrapper w;
25     int r;
26     w.u.p = read();
27     w.u.q = read();
28     w.tag = read();
29     r = evalWrap(w);
30     write(r);
31     return 0;
32 }

```

4.3.2 输出

```

1 >>> 5, 4, 3
2 [13]
3
4 >>> 1, 2, 15
5 [1]

```

```
6
7 >>> 0, 0, -1
8 [0]
```

4.3.3 说明

测试嵌套结构体与成员：Wrapper 内含 Inner 与 tag；evalWrap 按 tag 与 p, q 分段累加得分。

针对选做 **要求 4.1** 的同学；未实现时应提示无法翻译且不输出中间代码。

记号说明同 A-1。

4.4 E2-1

4.4.1 输入

```
1 int main() {
2     int g[2][3];
3     int sumCorner;
4     int scaled;
5     g[0][0] = 1;
6     g[0][1] = 2;
7     g[1][2] = 30;
8     sumCorner = g[0][0] + g[1][2];
9     scaled = g[0][1] * 3;
10    write(sumCorner);
11    write(scaled);
12    return 0;
13 }
```

4.4.2 输出

```
1 >>> Empty
2 [31, 6]
```

4.4.3 说明

测试二维数组下标与常数运算：对 g[2][3] 赋值后输出两路结果。无 read。

针对选做 **要求 4.2** 的同学；未实现高维数组时应提示无法翻译且不输出中间代码。

记号说明同 A-1。

4.5 E2-2

4.5.1 输入

```
1 int prefixSum(int arr[4], int n) {
2     int s;
3     int i;
4     s = 0;
5     i = 0;
6     while (i < n) {
7         s = s + arr[i];
8         i = i + 1;
9     }
10    return s;
11 }
12
13 int main() {
14     int a[4];
15     int s3;
16     int s4;
17     a[0] = 4;
18     a[1] = 7;
19     a[2] = 1;
20     a[3] = 9;
21     s3 = prefixSum(a, 3);
22     s4 = prefixSum(a, 4);
23     write(s3);
24     write(s4);
25     return 0;
26 }
```

4.5.2 输出

```
1 >>> Empty
2 [12, 21]
```

4.5.3 说明

测试一维数组作形参：prefixSum 求前 n 项和；主函数固定数组，输出前 3 项和与前 4 项和。

针对选做 **要求 4.2** 的同学；未实现数组参数时应提示无法翻译且不输出中间代码。

记号说明同 A-1。

4.6 E2-3

4.6.1 输入

```
1 int excludeSum(int arr[4], int skip) {
2     int sum;
3     int idx;
4     sum = 0;
5     idx = 0;
6     while (idx < 4) {
7         if (arr[idx] != skip) {
8             sum = sum + arr[idx];
9         }
10        idx = idx + 1;
11    }
12    return sum;
13 }
14
15 int main() {
16     int cube[2][2][2];
17     int line[4];
18     int t;
19     int cnt;
20     int s;
21     int i;
22     int j;
23     int k;
24     i = 0;
25     while (i < 4) {
26         line[i] = i * 2 + 5;
27         i = i + 1;
28     }
```

```

29     i = 0;
30     while (i < 2) {
31         j = 0;
32         while (j < 2) {
33             k = 0;
34             while (k < 2) {
35                 cube[i][j][k] = i * 9 + j * 3 + k;
36                 k = k + 1;
37             }
38             j = j + 1;
39         }
40         i = i + 1;
41     }
42     t = read();
43     cnt = 0;
44     i = 0;
45     while (i < 2) {
46         j = 0;
47         while (j < 2) {
48             k = 0;
49             while (k < 2) {
50                 if (cube[i][j][k] == t) {
51                     cnt = cnt + 1;
52                 }
53                 k = k + 1;
54             }
55             j = j + 1;
56         }
57         i = i + 1;
58     }
59     s = excludeSum(line, t);
60     write(cnt);
61     write(s);
62     return 0;
63 }

```

4.6.2 输出

```
1 >>> 9
2 [1, 23]
3
4 >>> 0
5 [1, 32]
```

4.6.3 说明

测试三维数组与一维数组形参：在 `cube[2][2][2]` 上统计值等于 `t` 的个数，并对固定一维表 `line` 调用 `excludeSum`（跳过等于 `t` 的元素求和）。

针对选做 **要求 4.2** 的同学；未实现时应提示无法翻译且不输出中间代码。

记号说明同 A-1。

5 结束语

若对本文档有任何疑议，可写邮件与陈泰霖助教联系，注意同时抄送给许畅老师。